# PA2 Overview Document

## CS 4120: Overview Documentation

### Programming Assignment 2: Implementing Syntactic Analysis

**Authors:** Fadi Ismail, Yoseph Endawoke, Charles Liggins, Temi Adebowale
**Date:** February 14, 2026

---

## 1. Metadata

- **Assignment:** Programming Assignment 2: Parser
- **Group members:**
  - Fadi Ismail (fmi4)
  - Yoseph Endawoke (ye38)
  - Charles Liggins (cll258)
  - Temi Adebowale (ta375)
- **How to run:**

```
# Build the compiler
./etac-build

# Parse a single .eta or .eti file
./etac --parse <filename>.eta
./etac --parse <filename>.eti

# Parse with output directory and source path
./etac --parse -D <outdir> --sourcepath <srcdir> <files...
>

# Lex a file (PA1 functionality still works)
```

```
./etac --lex <filename>.eta

# Run all PA2 tests and check against expected output
for f in eth/tests/pa2/*.eta eth/tests/pa2/*.eti; do ./eta
c --parse "$f" 2>/dev/null; done && PASS=0 && FAIL=0 && fo
r f in eth/tests/pa2/*.parsedsol; do base="${f%.parsedso
l}"; if diff -q "$base.parsed" "$f" > /dev/null 2>&1; then
PASS=$((PASS+1)); else echo "FAIL: $base"; FAIL=$((FAIL+
1)); fi; done && echo "$PASS passed, $FAIL failed" && rm -
f eth/tests/pa2/*.parsed
```

- **Main entry point:** `src/main.rs`
- **Environment tested:** macOS (Darwin 24.6.0), Linux VM (Ubuntu 24.04, Rust 1.93.0)

---

## 2. Summary

The most challenging aspect of this assignment was bridging two independent token systems: logos produces its own `Token` type with its own naming conventions, while LALRPOP requires an `extern` Token type declared in the grammar. We solved this with a thin adapter layer (`adapter.rs`) that iterates the logos lexer directly to obtain byte-offset spans (which LALRPOP needs) and maps each lexer token to its parser equivalent—without modifying any PA1 lexer code.

Key design decisions included:

- Using a MatchedIf/UnmatchedIf grammar split to resolve the dangling-else ambiguity, requiring If and While bodies to be `Box<Stmt>` rather than `Block`
- Redesigning the AST from PA1 to support multi-assignment (`x, y = f()`) via a unified `Stmt::Assign(Vec<AssignTarget>, Vec<Expr>)` variant
- Factoring identifier-started statements through an `IdentStmtRest` helper rule to resolve parsing ambiguity without backtracking
- Building an intermediate `SExp` tree (Atom | List) before rendering, which cleanly separates AST-to-S-expression conversion from formatting

A known limitation is that computed lvalue assignment (`f(x)[i] = e`) uses a placeholder in the AST. This is uncommon in practice and does not affect standard Eta programs.

---

# 3. Specification

We encountered several ambiguities or underspecified areas in the handout that required interpretation:

1. **Dangling else:** The specification does not prescribe a resolution. We use the standard MatchedIf/UnmatchedIf grammar split so that `else` always binds to the nearest `if`.
2. **Optional semicolons:** The grammar uses `";"?` after statements and `use` declarations, making semicolons optional everywhere. Programs with and without semicolons parse identically.
3. **Identifier-started statement ambiguity:** A statement beginning with an identifier could be an assignment, declaration, procedure call, or multi-assign. We factor these through an `IdentStmtRest` rule that delays the decision until enough tokens are consumed, then converts to the appropriate `Stmt` via `into_stmt()`.
4. **Operator precedence:** Not fully specified in the handout beyond the grammar. We use LALRPOP's `#[precedence]` and `#[assoc]` attributes with 8 levels: Or (lowest), And, Equality, Relational, Additive, Multiplicative, Unary, Postfix (highest). All binary operators are left-associative.
5. **Sized vs unsized arrays:** `Type::Array` stores an optional size expression. `int[3][]` produces `Array(Array(Int, None), Some(3))`. Dimensions are parsed left-to-right but built inside-out so the innermost type is the element type.
6. **S-expression format for errors:** On lex or parse error, output is `<line>:<col> error:<description>`. For parse errors we include the unexpected token name (e.g., `1:1 error:Unexpected token if`).

---

# 4. Design and Implementation

## 4.1 Architecture

**Module layout:**

| Module | File | Responsibility |
|---|---|---|
| CLI | `src/cli/cli.rs` | Command-line argument parsing using `clap`: `--lex`, `--parse`, `-D`, `--sourcepath` |
| I/O | `src/cli/io.rs` | File reading, output path computation, output writing |
| Formatter | `src/cli/formatter.rs` | Lexer output formatting (PA1) |
| Lexer | `src/lexer/token.rs` | Logos token definitions, `tokenize()`, `LexerExtras` |
| Callbacks | `src/lexer/callbacks.rs` | String/char literal parsing callbacks |
| AST | `src/parser/ast.rs` | AST node types, parser Token enum, grammar helper types |
| Grammar | `src/parser/eta.lalrpop` | LALRPOP grammar for Eta programs and interfaces |
| Adapter | `src/parser/adapter.rs` | Logos-to-LALRPOP token conversion with byte offsets |
| S-expression | `src/sexp.rs` | S-expression printer for Program and Interface ASTs |
| Pretty printer | `src/pretty.rs` | Source-code pretty printer (debugging utility) |
| Main | `src/main.rs` | Entry point, CLI dispatch, parse error formatting |

**Key interfaces:**

```
// Token adapter — converts logos tokens to LALRPOP-compatibl
e triples
pub fn lex_for_parser(source: &str)
    -> Result<Vec<(usize, Token, usize)>, LexError>

// Parser entry points (LALRPOP-generated)
ProgramParser::new().parse(tokens)   -> Result<Program, Parse
Error>
InterfaceParser::new().parse(tokens) -> Result<Interface, Par
seError>
```

```
// S-expression output
pub fn sexp_program(p: &Program) -> String
pub fn sexp_interface(iface: &Interface) -> String

// Core AST types
pub struct Program {
    pub uses: Vec<String>,
    pub items: Vec<TopLevelItem>,
}

pub struct Interface {
    pub decls: Vec<InterfaceDecl>,
}
```

**Data flow:**

```
Source File (.eta/.eti)
    │
    ▼
read_source_file() ──────────► String (source code)
    │
    ▼
lex_for_parser() ─────────────► Vec<(usize, Token, usize)>
    │                            (byte-offset triples)
    ▼                            or LexError { line, col, msg }
ProgramParser::parse()
  or InterfaceParser::parse() ► AST (Program or Interface)
    │
    ▼
sexp_program() / sexp_interface() ► String (S-expression)
    │
    ▼
write_output_file() ──────────► .parsed file
```

**Invariants:**

- Lex errors are caught before parsing and produce `line:col error:msg` output immediately
- Parse errors are converted from byte offsets to line:col via `byte_offset_to_line_col()`
- The adapter rejects tokens not in the Eta grammar (`Break`, `For`, `Let`) with an error
- `.eti` files are routed to `InterfaceParser`, `.eta` files to `ProgramParser`

## 4.2 Algorithms and Data Structures

**Core algorithms:**

The parser uses LALRPOP-generated LR(1) tables. The token adapter iterates the logos lexer directly (bypassing PA1's `tokenize()`) to produce the `(usize, Token, usize)` byte-offset triples that LALRPOP requires:

```
lex_for_parser(source) -> Result<Vec<(usize, Token, usize)>,
LexError>:
    lex = Token::lexer(source)
    tokens = []
    while let Some(result) = lex.next():
        span = lex.span()
        match result:
            Whitespace | Newline | Comment -> continue (skip)
            Error -> return LexError at lex.extras position
            token -> map to parser Token, push (span.start, t
oken, span.end)
    return Ok(tokens)

// Dangling-else resolution (grammar split):
MatchedStmt:
    if (Expr) MatchedStmt else MatchedStmt
    while (Expr) MatchedStmt
    Block | Assign

UnmatchedStmt:
```

```
    if (Expr) Stmt
    if (Expr) MatchedStmt else UnmatchedStmt
    while (Expr) UnmatchedStmt

// Identifier-started statement resolution:
IdentStmtRest:
    // After seeing an identifier, look at what follows:
    "(" args ")" rest    -> procedure call or multi-assign
    "[" indices "]" rest -> array index assignment
    ":" Type rest        -> declaration
    "," more_targets "=" exprs -> multi-assign
    "=" exprs            -> simple assignment
    // Converted to Stmt via into_stmt()
```

**Data structures:**

- `Token` (parser): Enum with 30+ variants matching the LALRPOP extern block. Data-carrying variants: `Identifier(String)`, `IntLiteral(i64)`, `CharLiteral(i64)`, `StringLiteral(String)`.
- `Program`: `{ uses: Vec<String>, items: Vec<TopLevelItem> }` where `TopLevelItem` is `Func(FuncDef)` or `Global(GlobalDecl)`.
- `Stmt`: Enum with four variants: `If`, `While`, `Block`, and `Assign`. If and While hold `Box<Stmt>` for their bodies (not `Block`) to support the dangling-else grammar split. `Assign` holds `Vec<AssignTarget>` and `Vec<Expr>` to uniformly represent declarations, assignments, procedure calls, and multi-assignments.
- `AssignTarget`: `Discard | Var(String) | Decl(String, Type) | ArrayIndex(String, Vec<Expr>)`.
- `Type`: `Int | Bool | Array(Box<Type>, Option<Box<Expr>>)`. The optional expression stores sized array dimensions.
- `SExp`: Intermediate representation for S-expression output — either `Atom(String)` or `List(Vec<SExp>)`.
- `IdentStmtRest` / `DeclSuffix`: Grammar-internal helper types that resolve identifier-started statement ambiguity. They are consumed during parsing via `into_stmt()` and never appear in the final AST.

**Tradeoffs:**

- *Unified* `Stmt::Assign` *vs separate variants:* We chose a single `Assign(Vec<AssignTarget>, Vec<Expr>)` variant over separate `VarDecl`, `Assignment`, `Proc`, and `Return` variants. This is more uniform and directly supports multi-assignment, but makes pattern matching slightly less obvious for simple cases (e.g., a procedure call is `Assign([], [FuncCall(...)])`).
- `Box<Stmt>` *for If/While bodies vs* `Block`*:* Required by the dangling-else resolution. A single statement like `if (x) y = 1` has a non-block body, so the grammar must accept `Stmt`, not just `Block`.
- *Adapter iterates logos directly vs reusing* `tokenize()`*:* We chose direct iteration to get byte offsets without modifying the PA1 lexer. The cost is a second Token enum and mapping code, but PA1 code is untouched.
- *Intermediate* `SExp` *tree vs direct string building:* Building an `SExp` tree first and then rendering it separates conversion logic from indentation/formatting. This made the printer easier to write and debug, at the cost of an extra allocation pass.

## 4.3 Implementation Strategy and Collaboration

**Implementation strategy:** We used a layered approach, building on top of the working PA1 lexer:

1. First, redesigned the AST to match the LALRPOP grammar requirements
2. Created `build.rs` to invoke LALRPOP at compile time
3. Fixed the existing LALRPOP grammar (import paths, `Box::new` on recursive types, semicolon handling)
4. Built the token adapter to bridge logos and LALRPOP
5. Wired up the `-parse` CLI flag and file I/O pipeline
6. Implemented the S-expression printer and verified against spec examples
7. Added `-sourcepath` flag and `.eti` interface file support
8. End-to-end testing with custom tests and spec examples

This approach worked well because each layer could be tested independently before integration.

**Challenges:**

- **AST redesign:** The original PA1 AST had types that didn't match the grammar's needs (separate statement variants, `Block` only bodies). Restructuring this while keeping the grammar working required coordinated changes across `ast.rs` and `eta.lalrpop`.
- **LALRPOP byte offsets:** LALRPOP requires `(usize, Token, usize)` triples with byte offsets, but our PA1 lexer only tracked line/col. The adapter pattern solved this without touching PA1 code.
- **Dangling else:** Getting the MatchedIf/UnmatchedIf split correct required careful attention to ensure all statement forms were covered in both rules.
- **Array dimension ordering:** `int[3][]` must produce `Array(Array(Int, None), Some(3))` — dimensions are parsed left-to-right but the type is built inside-out. Getting the reversal right in `into_stmt()` took debugging.

**Code reuse:** The PA1 lexer was reused without modification via the adapter layer. The CLI module (`cli.rs`, `io.rs`) was extended with new flags but the existing code was unchanged.

---

# 5. Testing

## 5.1 Test Plan

We did not assume that the staff-provided tests were complete, so we designed additional tests directly from the Eta syntactic specification. Our testing strategy targeted every major grammar construct individually (functions, globals, if/else, while, return, types, operators, arrays, function calls, `length`, discard `_`), realistic multi-feature programs, and both parse errors (invalid syntax) and lex errors propagated through `--parse` (malformed literals). We verified against the spec's exact S-expression examples on page 7 of the handout to ensure output format correctness.

Testing was performed incrementally. As new grammar rules were implemented, we added small, focused tests to isolate that behavior before relying on larger

integration-style programs. This approach helped localize bugs quickly and ensured that regressions were caught early. PA1 lexer tests were re-run as a regression check.

## 5.2 Test Suite Organization

- **Where tests live:** All PA2 tests are in `eth/tests/pa2/`, with each test consisting of a `.eta`/`.eti` input file and a matching `.parsedsol` expected-output file. An `ethScript` file defines the test harness configuration.
- **How to run tests:**

```
# Build and run all PA2 tests with pass/fail summary
./etac-build && for f in eth/tests/pa2/*.eta eth/tests/pa
2/*.eti; do \\
  ./etac --parse "$f" 2>/dev/null; done && \\
PASS=0 && FAIL=0 && for f in eth/tests/pa2/*.parsedsol; do
\\
  base="${f%.parsedsol}"; \\
  if diff -q "$base.parsed" "$f" > /dev/null 2>&1; then \\
    PASS=$((PASS+1)); \\
  else echo "FAIL: $base"; FAIL=$((FAIL+1)); fi; done &&
\\
echo "$PASS passed, $FAIL failed" && rm -f eth/tests/pa2/
*.parsed

# Run via the course test harness
cargo build --release
eth -compilerpath target/release -testpath eth/tests/pa2 e
thScript
```

- **Categories tested (27 tests):**
  - Valid programs (17): empty program, functions, use declarations, globals, if/else, while, return, types, all operators, `>>`, assignments, literals, array init, function calls, `length()`, discard `_`, complex multi-function program

- Interface files (2): `.eti` files with single and multi-return function signatures
- Parse errors (6): missing brace, missing paren, invalid top-level token, double assign, missing type, incomplete expression
- Lex errors through `-parse` (2): unterminated string, empty character literal

### 5.3 Results

- **Systems tested on:** All tests were run in the provided Linux VM environment (Ubuntu 24.04, Rust 1.93.0) using the course test harness, as well as locally on macOS during development.
- **Passing tests:** At the time of submission, all 27 PA2 tests pass. All 23 PA1 lexer tests pass as a regression check.
- **Failing tests:** None.
- **Unit tests:** Rust unit tests exist for the S-expression printer (`sexp_tests.rs`) and pretty printer (`pretty_tests.rs`), run via `cargo test`.

---

## 6. Work Plan

The work was divided by major components with clear dependencies:

| Team Member | Responsibilities |
|---|---|
| **Temi** | AST redesign to match grammar, `build.rs` setup, token adapter (`adapter.rs`), `--parse` CLI integration, parser test suite design and implementation |
| **Fadi** | S-expression printer (`sexp.rs`), pretty printer (`pretty.rs`), `--sourcepath` CLI flag, array dimension fix in AST, S-expression and pretty printer unit tests |
| **Yoseph** | Lexer implementation and fixes from PA1, Grammar Design |
| **Charles** | AST Design and Grammar Structure |

---

## 7. AI Usage

We used AI tools (Claude, GitHub Copilot) in the following ways:

- **Debugging assistance:** When encountering confusing LALRPOP grammar conflicts and logos-to-LALRPOP adapter issues, we used AI to explain error messages and suggest fixes
- **Documentation:** AI helped draft initial versions of code comments and this overview document
- **Test brainstorming:** We asked AI to suggest edge cases we might have missed (e.g., "what are common parser edge cases for dangling else?")
- **Rust syntax:** Copilot provided autocomplete suggestions for boilerplate code (match arms, struct definitions)

AI was helpful for accelerating routine tasks and catching blind spots in test coverage. However, we found that AI suggestions for the core parser and grammar logic often missed nuances of the Eta specification, so all grammar and S-expression code was written and verified manually against the spec.

---

## 8. Known Problems

None

---

## 9. Comments

None